

Une architecture de données OLTP · OLAP · NoSQL

Trois besoins qu'aucune base ne satisfait seule — trois systèmes spécialisés, un bus d'événements, une seule source de vérité.

Encaisser

des millions de paiements par jour sans jamais en perdre un

OLTP · PostgreSQL

Analyser

des années d'historique en secondes, sur toutes les dimensions

OLAP · star schema

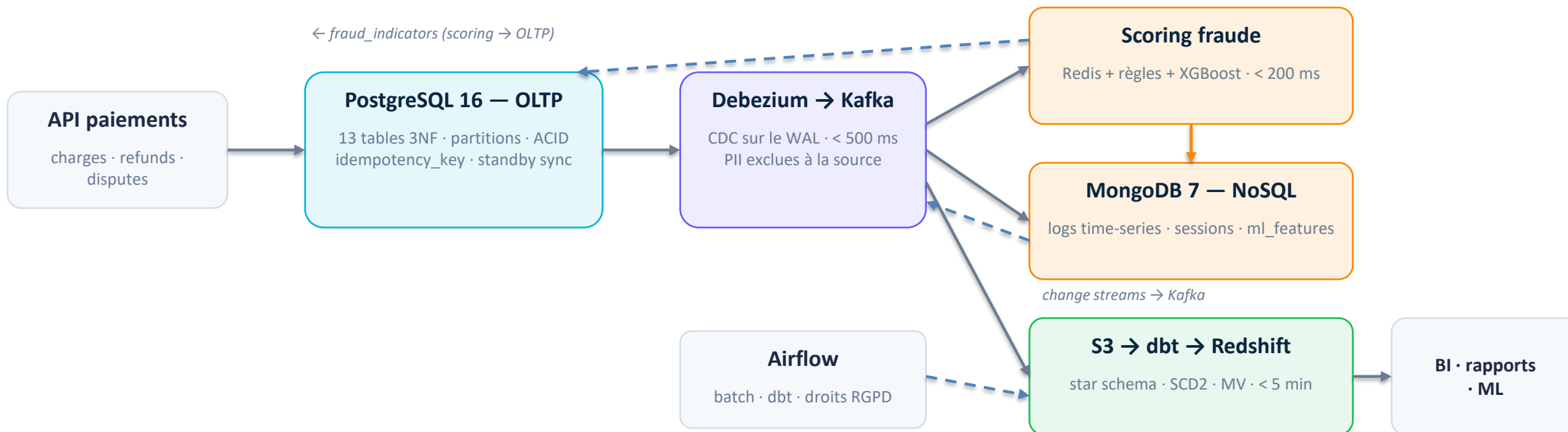
Exploiter

logs, sessions et features de fraude dont le schéma change chaque semaine

NoSQL · MongoDB

Données synthétiques, volumes de démo. Tout ce qui suit est exécuté — en local et en production sur AWS.

Chaque système fait ce qu'il fait le mieux — Kafka les relie



Les faits d'argent en SQL

Source de vérité unique, ACID, RPO 0 / RTO < 60 s.

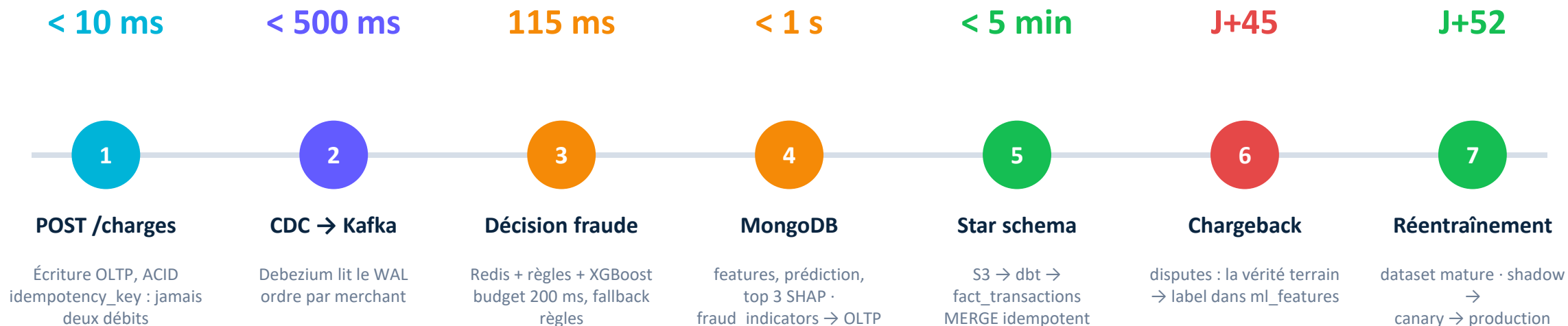
Le contexte en documents

Schéma flexible mais validé (\$jsonSchema), TTL, sharding.

L'analyse en colonnes

Grain = 1 transaction, dimensions SCD Type 2, 3 niveaux de pré-agrégation.

La vie d'une transaction : sept étapes, une garantie à chacune



- ▶ **Chaque saut est idempotent** : clé unique, upsert, MERGE, manifeste COPY. Kafka est at-least-once ; l'idempotence en fait un exactly-once effectif.
- ▶ **Si Kafka tombe**, l'OLTP continue (CDC asynchrone). Si l'entrepôt tombe, S3 accumule et rattrape. Si le scoring tombe, les règles dures décident : on ne bloque jamais un paiement parce qu'un modèle est lent.
- ▶ **Le chargeback à J+45 est la vérité terrain de la fraude** : une architecture qui ne sait pas le rattacher à la transaction d'origine ne peut pas améliorer son modèle.

La conformité est dans le code, le ML a une boucle de feedback

Conformité — quatre lignes qui valent un chapitre

Audit immuable (PostgreSQL)

```
CREATE TRIGGER trg_audit_immutable
BEFORE UPDATE OR DELETE ON audit_logs ...
RAISE EXCEPTION 'append-only'
```

PII exclues du CDC (Debezium)

```
"column.exclude.list":
  "customers.email, transactions.ip_address"
```

IP tronquée pour les analystes (vue)

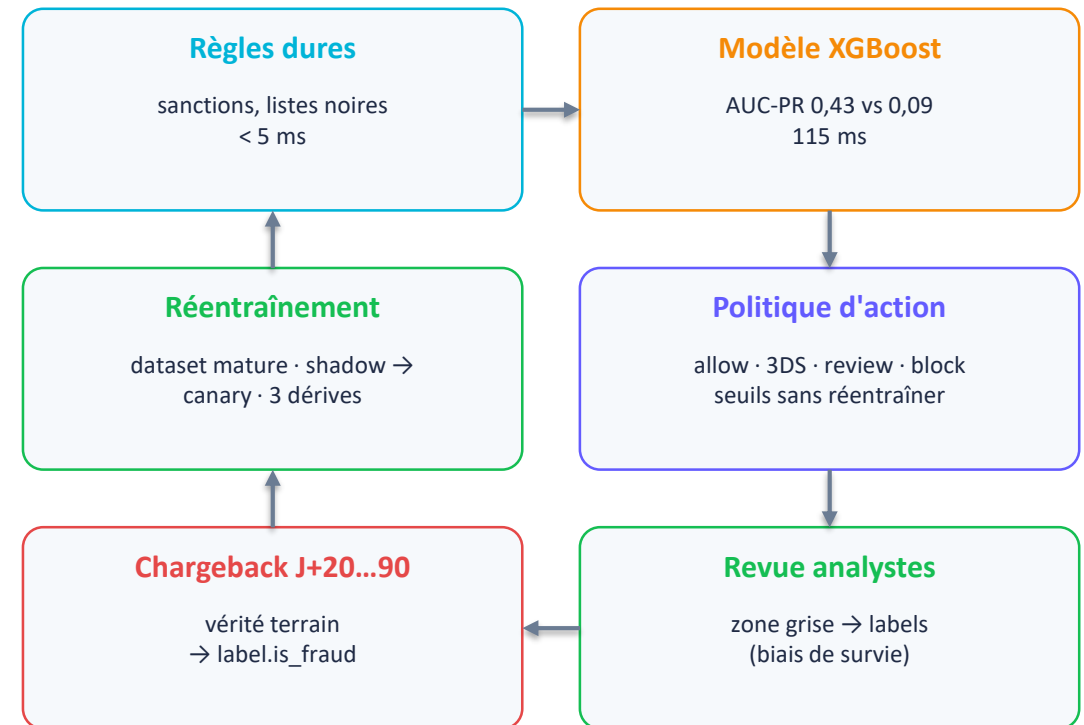
```
host(set_masklen(ip_address::cidr, 24))
-- 185.12.44.21 → 185.12.44.0
```

Contrôle PII bloquant (dbt / DAG)

```
dim_customer.email_hash:
  not_accepted_values: ['%@%'] # DAG rouge
```

Aucun numéro de carte nulle part (tokenisation) · droit à l'oubli = un DAG qui détache l'identité sans supprimer les faits comptables (10 ans).

ML — la boucle qui apprend des chargebacks



Une transaction bloquée n'est jamais contestée : sans revue humaine, le modèle ne réapprendrait que ses propres erreurs.

En production sur AWS — le pipeline tourne chaque nuit

Infrastructure (Terraform, 19 ressources, eu-north-1)



Rôle d'instance pour S3 — aucune clé AWS · security groups sur une seule IP · secrets générés par Terraform, hors dépôt

DAG stripe_pipeline — 14 tâches, ≈ 25 s



Contrôles inter-systèmes bloquants : fact OLAP = transactions OLTP = ml_features Mongo · aucun email en clair · une version SCD2 courante par merchant.

25 s

run complet, chaque nuit à 02:00 UTC

10 / 10

contrôles de cohérence verts

30

requêtes exécutées, résultats archivés

0,43

AUC-PR du modèle de fraude (baseline 0,09)

Écart assumé

L'entrepôt de production est PostgreSQL sur RDS, pas Redshift : même star schema, même vue matérialisée, mêmes 10 requêtes. Le passage à Redshift est un changement de DDL et de connexion, pas de conception.

github.com/emelineroblot/jedha-stripe-project · vidéo du pipeline en production